Domain-3 Security Architecture and Engineering

You may find this domain to be more technical than others, and if you have experience working in a security engineering role you likely have an advantage; if not, allocate extra time to this domain to ensure you have a good understanding of the topics; domain 3 is weighted around 13%

- **Advanced Encryption Standard (AES)**: uses the Rijndael symmetric algorithm and is the US gov standard for the secure exchange of sensitive but unclassified data; AES uses key lengths of 128, 192, and 256 bits, and a fixed block size of 128 bits, achieving a higher level of security than the older DES algorithm
- **Algorithm**: a mathematical function that is used in the encryption and decryption process; can be simple or very complex; also defined as a set of instructions by which encryption and decryption is done
- **Argon2**: a secure key derivation and password hashing algorithm designed to protect against brute-force and side-channel attacks; it was the winner of the Password Hashing Competition in 2015 and is considered highly secure and efficient, especially for systems requiring robust password protection
- **ASLR**: Address space layout randomization (ASLR) is a memory-protection process for operating systems (OSes) that guards against buffer-overflow attacks by randomizing the location where system executables are loaded into memory
- **Block Mode Encryption**: using fixed-length sequences of input plaintext symbols as the unit of encryption
- **Block cipher**: method of encrypting text that produces ciphertext, where a cryptographic key and algorithm are applied to a block of data at once/as a group, instead of one bit at a time; takes a number of bits and encrypts them in a single unit, padding the plaintext to achieve a multiple of the block size; the Advanced Encryption Standard (AES) algorithm uses 128-bit blocks
- **Cipher**: always meant to hide the true meaning of a message; always secret; types of ciphers include transposition, substitution, stream, and block
- **Ciphertext**: altered form of a plaintext message so as to be unreadable for anyone except the intended recipients (it's a secret)
- **Cleartext**: any information that is unencrypted, although it might be in an encoded form that is not easily human-readable (such as base64 encoding)
- **Cloud Controls Matrix (CCM)**: Cloud Security Alliance (CSA) framework designed to provide security principles to guide cloud vendors and assist prospective cloud customers in assessing the risks of cloud usage

- **Cloud Security Posture Management (CSPM)**: identifies and remediates risk by automating visibility, uninterrupted monitoring, threat detection, and remediation workflows searching for misconfigurations across cloud environments and infrastructure such as Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS); these tools can also perform incident response, remediation recommendations, and compliance monitoring
- **Code**: cryptographic systems of symbols that operate on words or phrases and are sometimes secret, but don't always provide confidentiality
- **Collision**: occurs when a hash function generates the same output for different inputs
- **Cryptanalysis**: study of techniques for attempting to defeat cryptographic methods and generally information security services; Cryptanalysis is the process of transforming or decoding communications from non-readable to readable format without having access to the real key; two major types of cryptanalysis: cryptanalytic attacks, and cryptographic attacks
- **Cryptanalytic attack**: attack with a primary goal of deducing the key
- **Cryptographic Hash function**: process or function that transforms an input plaintext into a unique value called a hash (or hash value); note that they do not use cryptographic algorithms, as hashes are one-way functions where it's infeasible to determine the plaintext; Message digests are an example of cryptographic hash
- **Cryptography**: study of/application of methods to secure the meaning and content of messages, files etc by disguise, obscuration, or other transformations
- **Cryptosystem**: complete set of hardware, software, communications elements and procedures that allow parties to communicate, store or use info protected by cryptographic means; includes algorithm, key, and key management functions
- **Cryptovariable**: parameter associated with a particular cryptographic algorithm; e.g. block size, key length and number of iterations
- **Cyber-physical systems**: systems that use 'computational means' to control physical devices
- **Decoding**: the reverse process from encoding, converting the encoded message back to plaintext format
- **Decryption**: the reverse process from encryption
- **Elliptic-curve cryptography (ECC)**: a newer mainstream asymmetric algorithm, is normally 256 bits in length (a 256-bit ECC key is equivalent to a 3072-bit RSA key), making it securer and able to offer stronger anti-attack capabilities

- **Encoding**: action of changing a message or set of info into another format through the use of code; unlike encryption, encoded info can still be read by anyone with knowledge of the encoding process
- **Encryption**: process and act of converting the message from plaintext to ciphertext (AKA enciphering)
- **Enterprise Security Architecture**: methods to ensure security is aligned with org goals and objectives to protect critical components (people, process, and technology)
- **Factoring attack**: in terms of the test, only the RSA algorithm uses factoring as the hard math problem; so if you see factoring attack, think RSA
- **Frequency analysis**: form of cryptanalysis that uses frequency of occurrence of letters, words or symbols in the ciphertext as a way of reducing the search space
- **Hybrid encryption system**: a method that combines the speed and efficiency of symmetric-key cryptography with the secure key exchange capabilities of asymmetric (public-key) cryptography; most practical implementations of public-key cryptography today use this hybrid approach
- **International Data Encryption Algorithm (IDEA)**: IDEA is a form of symmetric key block cipher encryption that uses a 128-bit key and operates on 64-bit blocks; it encrypts a 64-bit block of plaintext into a 64-bit block of ciphertext, and the input plaintext block is divided into four sub-blocks of 16 bits each
- **Initialization Vector (IV)**: a random string of bits (aka a nonce) that is XORed with a message, reducing predictability and repeatability; size of the IV varies by algorithm but normally is the same length as the block size of the cipher (or as large as the encryption key); IV is the cryptographic version of a random number
- **Key**: the input that controls the operation of the cryptographic algorithm, determining the behavior of the algorithm and permits the reliable encryption and decryption of the message
- **Key clustering**: weakness in cryptography where a plain-text message generates identical ciphertext messages when using the same algorithm but using different keys
- **Key escrow**: process by which keys (asymmetric or symmetric) are placed in a trusted storage agent's custody, for later retrieval
- **Key generation**: the process of creating a new encryption/decryption key
- **Key pair**: matching set of one public and one private key

- **Key recovery**: process of reconstructing an encryption key from the ciphertext alone; if there is a workable key recovery system, it means the algorithm is not secure
- **Key space**: represents the total number of possible values of keys in a cryptographic algorithm or password; keyspace = 2^n (where n = number of bits), so 4 bits = 16 keys, 8 bits = 256 keys
- **Lattice-based Cryptography**: Lattice-based cryptography leverages complex grids or constructions known as lattices for the purpose of encryption and decryption; involves mathematical problems that remain hard to solve even with the enhanced computational power of quantum computing
- **Microcontroller**: similar to system on a chip (SoC), consists of a CPU, memory, IO devices, and non-volatile storage (e.g. flash or ROM/PROM/EEPROM); think Raspberry Pi or Arduino
- **Mobile device deployment models**: that cover allowing or providing mobile devices for employees include: BYOD (Bring Your Own Device), COPE (Company Owned/Personally Enabled), CYOD (Choose Your Own Device), and COBO (Company Owned/Business Only); also consider VDI and VMI options
- **Mobile device deployment policies**: should address things like data ownership, support ownership, patch and update management, security product management, forensics, privacy, on/offboarding, adherence to corporate policies, user acceptance, legal concerns, acceptable use policies, camera/video, microphone, Wi-Fi Direct, tethering and hot spots, contactless payment methods, and infrastructure considerations
- **Multi-state systems**: certified to handle data from different security classifications simultaneously
- **One-time pad**: series of randomly generated symmetric encryption keys, each one to be used only once by the sender and recipient; to be successful, the key must be generated randomly without any known pattern; the key must be at least as long as the message to be encrypted; the pads must be protected against physical disclosure and each pad must be used only one time, then discarded
- **Out-of-band**: transmitting or sharing control information (e.g. encryption keys and crypto variables) by means of a separate and distinct communications path, channel, or system
- **Password-Based Key Derivation Function 2 (PBKDF2)**: securely derives cryptographic keys from passwords; by applying salting and key stretching (through multiple hashing iterations), PBKDF2 transforms a password into a cryptographic key that can be used for encrypting data or securely storing passwords; this process makes it much harder for attackers to guess or brute-

force the password, as it increases the computational work required to test each possible password, improving resistance against attacks

- **Pepper**: a large constant number used to increase the security of the hashed password further; it is stored outside of the database holding the hashed passwords
- **Personal electronic device (PED)** security features can usually be managed using mobile device management (MDM) or unified endpoint management (UEM) solutions, including device authentication, full-device encryption, communication protection, remote wiping, communication protection, device lockout, screen locks, GPS and location services, content management, app control, push notification management, third-party app store control, rooting/jailbreaking, credential management and more
- **Plaintext**: message or data in its readable form, not turned into a secret
- **Post-Quantum Cryptography**: development of new types of cryptographic approaches that can be implemented using conventional computing, and is resistant to quantum computing attacks; note that Lattice-based cryptography is resistant to most, but not all, quantum attacks (also see my article on quantum computing threats and opportunities)
- **Remote attestation**: feature of the TPM (Trusted Platform Module) that creates a hash value from the system configuration to confirm the integrity of the configuration
- **RTOS**: real-time operating system (RTOS) is an operating system specifically designed to manage hardware resources and run applications with precise timing and high reliability; they are designed to process data with minimum latency; an RTOS is often stored on ROM; they use deterministic timing, meaning tasks are completed within a defined time frame and is designed to operate in a hard (i.e. missing a deadline can cause system failure) or soft (missing a deadline degrades performance but is not catastrophic) real-time condition
- **Salting**: adds additional bits to a password before hashing it, and helps thwart rainbow attacks; algorithms like Argon2, bcrypt, and PBKDF2 add salt and repeat the hashing function many times; salts are stored in the same database as the hashed password
- **Salting vs key stretching**: salting adds randomness and uniqueness to each password before hashing, which reduces the effectiveness of rainbow table attacks; key stretching makes the hashing process deliberately slow, making it much more challenging for attackers to crack passwords using brute-force or precomputed tables; common password hashing algorithms that use key stretching include PBKDF2, bcrypt, and scrypt

- **SDx**: software-defined everything refers to replacing hardware with software using virtualization; includes virtualization, virtualized software, virtual networking, containerization, serverless architecture, IaC, SDN, VSAN, software-defined storage (SDS), VDI, VMI SDV, and software-defined data center (SDDC)
- **Session key**: a symmetric encryption key generated for one-time use; usually requires a key encapsulation approach to eliminate key management issues
- **Sherwood Applied Business Security Architecture (SABSA)**: Enterprise Security Architecture based on a risk-driven model based on business requirements for security
- **Static Environments**: apps, OSs, hardware, or networks that are created/configured to meet a particular need or function are set to remain unaltered; static environments, embedded systems, network-enabled devices, edge, fog, and mobile devices need security management that may include network segmentation, security layers, app firewalls, manual updates, firmware version control, wrappers, and control redundancy/diversity
- **Stream mode encryption**: system using a process that treats the input plaintext as a continuous flow of symbols, encrypting one symbol at a time; usually uses a streaming key, using part of the key as a one-time key for each symbol's encryption
- **Stream cipher**: a symmetric key cipher where plaintext digits are combined with a pseudorandom cipher digit stream; each plaintext digit is encrypted one at a time with the corresponding digit of the keystream, to give a ciphertext stream
- **Substitution cipher**: uses an encryption algorithm to replace each character or bit of the plaintext message with a different character; one of the earliest substitution ciphers was developed by Julius Caesar, known as the "Caesar cipher"
- **Symmetric encryption**: process that uses the same key (or a simple transformation of it) for both encryption/decryption
- **The Open Group Architecture Framework (TOGAF)**: Enterprise Security Architecture that provides for rapid and iterative development, defining business goals and aligning them with architecture objectives
- **Transposition cipher**: encryption/decryption process using transposition
- **Trust and Assurance**: trust is the presence of a security mechanism or capability; assurance is how reliable the security mechanism(s) are at providing security
- **VESDA**: very early smoke detection process (air sensing device brand name)

- **Work factor**: (AKA Work function) is a way to measure the strength of a cryptography system, measuring the effort in terms of cost/time to decrypt messages; amount of effort necessary to break a cryptographic system using a brute-force attack, measured in elapsed time
- **Zachman**: Enterprise Security Architecture based on 2-d table of what, how, when who, where, why; and identification, definition, representation, specification, configuration, and installation
- **Zero-knowledge proof**: one person demonstrates to another that they can achieve a result that requires sensitive info without actually disclosing the sensitive info

3.1 Research, implement, and manage engineering processes using secure design principles (OSG-10 Chpts 1,8,9,16)

- Standard **secure design principles** are:
    - Threat modeling
    - Least privilege
    - Defense in depth
    - Secure defaults
    - Fail securely
    - Keep it simple
    - Separation of duties
    - Zero trust
    - Shared responsibility
    - Privacy by design
- 3.1.1 Threat Modeling
    - **Threat modeling**: (see Domain 1), a security process where potential threats are identified, categorized, and analyzed; it can be performed as a proactive measure during design and development or as a reactive measure once a product has been deployed
        - Threat modeling identifies the potential harm, the probability of occurrence, the priority of concern, and the means to eradicate or reduce the threat
        - Threat modeling commonly involves decomposing the app to understand it and how it interacts with other components or users; identifying and ranking threats allows potential threats to be prioritized; identifying how to mitigate those threats finishes the process

        - For the exam, you should know the four primary threat modeling methods (DREAD, OCTAVE, STRIDE, and Trike)
- 3.1.2 Least Privilege
    - As noted in Domain 2, Least privilege states that subjects are granted only the privileges necessary to perform assigned work tasks and no more; this concept extends to data, software, and system design to reduce the surface, scope, and impact of any attack
        - Limiting and controlling privileges based on this concept protects confidentiality and data integrity
- 3.1.3 Defense in Depth
    - **Defense in Depth**: AKA layering, is the use of multiple controls in a series, where a single failed control should not result in exposure of systems or data; layers should be used in a series (one after the other), NOT in parallel (note that layered security does also involve parallel controls (e.g., both a firewall and IDS monitoring the same traffic); here the "series" concept refers multiple sequential barriers an attacker must overcome, not that controls cannot operate concurrently)
        - When you see the terms like levels, multilevel, layers, classifications, zones, realms, compartments, protection rings etc think about Defense in Depth
- 3.1.4 Secure defaults
    - **Secure defaults**: refer to the practice of configuring systems, applications, and devices with the most secure settings enabled by default, minimizing risk without requiring user intervention; when you think about defaults, consider how something operates brand new, just turned over to you by the vendor
        - e.g. wireless router default admin password, or firewall configuration requiring changes to meet an organization's needs
- 3.1.5 Fail securely
    - **Fail securely**: if a system, asset, or process fails, it shouldn't reveal sensitive information, or be less secure than during normal operation; failing securely could involve reverting to defaults
    - Physical vs digital failure table

State	Digital	Physical
Fail-Open	maintain system availability	protect people
Fail-Safe	maintain confidentiality/integrity	protect people
Fail-Closed	maintain c/i	protect asset
Fail-Secure	maintain c/i	protect asset

- 3.1.6 Separation of duties (SoD)
    - **Separation of duties (SoD)**: separation of duties (SoD) and responsibilities ensures that no single person has total control over a critical function or system; SoD is a process to minimize opportunities for misuse of data or environment damage; separation of duties helps prevent fraud
        - e.g. one person sells tickets, another collects tickets and restricts access to ticket holders in a movie theater
- 3.1.7 Keep it simple and small
    - **Keep it simple**: AKA keep it simple, stupid (KISS), this concept is the encouragement to avoid over-complicating the environment, organization, or product design
- 3.1.8 Zero Trust or trust but verify
    - **Zero Trust**: a security model that assumes no user, device, or system is inherently trusted, even inside the network; it enforces continuous verification, least privilege access, and strict access controls for every request; "assume breach"; a security concept and alternative of the traditional (castle/moat) approach where nothing is automatically trusted; instead each request for activity or access is assumed to be from an unknown and untrusted location until otherwise verified
    - **Trust but verify**: based on a Russian proverb, and no longer sufficient; it's the traditional approach of trusting subjects and devices within a company's security perimeter automatically, leaving an org vulnerable to insider attacks and providing intruders the ability to easily perform lateral movement
    - "Never trust, always verify" replaces "trust but verify" as a security design principle by asserting that all activities by all users/entities must

be subject to control, authentication, authorization, and management at the most granular level possible
    - Goal is to have every access request authenticated, authorized, and encrypted prior to access being granted to an asset or resource
    - See my article on an Overview of Zero Trust Basics
- 3.1.9 Privacy by design
    - **Privacy by design (PbD)**: a guideline to integrate privacy protections into products during the earliest design phase rather than tacking it on at the end of development
        - Same overall concept as "security by design" or "integrated security" where security is an element of design and architecture of a product starting at initiation and continuing through the software development lifecycle (SDLC)
        - There are 7 recognized principles to achieve privacy by design:
            - Proactive, preventative: think ahead and design for things that you anticipate might happen
            - Default setting: make private by default, e.g. social media app shouldn't share user data with everybody by default
            - Embedded: build privacy in; don’t add it later
            - Full functionality, positive-sum: achieve both security and privacy, not just one or the other
            - Full lifecycle, end-to-end protection: privacy should be achieved before, during and after a transaction; part of this is securely disposing of data when it is no longer needed
            - Visibility, transparency, open: publish the requirements and goals; audit them and publish the findings
            - Respect, user-centric: involve end users, providing the right amount of information for them to make informed decisions about their data
- 3.1.10 Shared responsibility
    - **Shared responsibility**: the security design principle that organizations do not operate in isolation
        - Everyone in an organization has some level of security responsibility
        - The job of the CISO and security team is to establish & maintain security
        - The job of regular employees to perform their tasks within the confines of security

        - The job of the auditor is to monitor the environment for violations
        - Because we participate in shared responsibility we must research, implement, and manage engineering processes using secure design principles
        - Shared responsibility also refers to the division of security responsibilities between a cloud service provider (CSP) or managed service providers (MSPs) and the customer (organization)
        - When working with third parties, especially with cloud providers, each entity needs to understand their portion of the shared responsibility of performing work operations and maintaining security; this is often referenced as the **cloud shared responsibility model**
        - The CSP handles the security of the cloud, while the customer is responsible for security in the cloud, especially things like data, identity, and access, and configuration
- 3.1.11 Secure access service edge
    - **Secure Access Service Edge (SASE)**: a cloud-delivered framework that brings together networking and security functions into a unified platform, integrating capabilities like Software-Defined Wide Area Networking (SD-WAN), Secure Web Gateway (SWG), Cloud Access Security Broker (CASB), Firewall-as-a-Service (FWaaS), and Zero Trust Network Access (ZTNA); SASE aims to:
        - Address traditional or legacy security mode and architecture limitations by eliminating blind spots and maintaining enterprise-wide protection via continuous monitoring of user behavior and network conditions
        - Securing remote access associated with remote and hybrid work models by providing granular access controls and identity-based authentication, where every user and device must be authenticated and authorized for resources access
        - Enhancing cloud adoption by integrating on-premise and cloud and providing control visibility via cloud
        - Brings security and networking closer to user/devices by leveraging edge computing which improves performance/reduces latency, and ensures consistent security treatment no matter where users are geographically
        - Table comparing SASE to traditional perimeter model:

Aspect	Traditional Perimeter-Based Model	Secure Access Service Edge (SASE)
Security Philosophy	"Castle-and-moat" — strong perimeter, trusted internal network	Zero Trust — **never trust, always verify**; identity-based security
Network Architecture	Centralized; everything routes through data center or HQ perimeter	Distributed; **cloud-native and edge-delivered** services
Access Location Assumption	Assumes users and resources are inside the network perimeter	Assumes users, devices, and data are **everywhere** (remote, cloud)
Traffic Routing	Backhauls remote traffic to a central location for inspection	Connects users directly to services via **nearest cloud edge**
Key Technologies	Firewalls, VPNs, IDS/IPS, on-prem proxies	**SD-WAN, ZTNA, CASB, SWG, FWaaS** integrated into a cloud solution
Scalability	Difficult to scale; hardware-dependent	**Easily scalable**, cloud-delivered model
Visibility & Control	Limited to on-premises infrastructure	**Centralized visibility** across users, apps, and devices globally
Latency & Performance	Higher latency (especially for remote/cloud access)	**Lower latency**, local breakout to nearest edge node
Cloud & SaaS Integration	Not designed for cloud-native environments	**Built for cloud and SaaS** access and protection
User & Device Trust Model	Implicit trust inside network	**Continuous verification** based on identity, device posture, etc.

3.2 Understand the fundamental concepts of security models (e.g. Biba, Star Model, Bell-LaPadula) (OSG-10 Chpt 8)

- Security models:
    - Intended to provide an explicit set of rules that a computer can follow to implement the fundamental security concepts, processes, and procedures of a security policy
    - Provide a way for a designer to map abstract statements into a security policy prescribing the algorithms and data structures necessary to build hardware and software
    - Enable people to access only the data classified for their clearance level
- **State machine model**: ensures that all instances of subjects accessing objects are secure
- **Information flow model**: designed to prevent unauthorized, insecure, or restricted information flow; the Information Flow model is an extension of the state machine concept and serves as the basis of design for both the Biba and Bell-LaPadula models
- **Noninterference model**: a Non-Interference Security Model ensures that actions taken at a higher security level do not influence or interfere with the observable behavior at a lower security level; prevents the actions of one subject from affecting the system state or actions of another subject
- **Bell-LaPadula**: Model was established in 1973; the goal is to ensure that information is exposed only to those with the right level of classification
    - Focus is on *confidentiality*
    - **Simple property**: "No read up"
    - **Star (*) property**: "No write down" (AKA confinement property)
    - **Strong Star Security Property**: subject can read/write only in their own layer of secrecy
    - **Discretionary Security Property**: uses an access matrix (need to know in order to access)
    - Doesn't address covert channels
- **Biba**: Released in 1977, this model was created to supplement Bell-LaPadula
    - Focus is on *integrity*
    - **Simple Integrity Property**: "No read down" (for example, users with a Top Secret clearance can’t read data classified as Secret)
    - **Star (*) Integrity Property**: "No write up" (for example, a user with a high integrity clearance can’t write data to files classified as low integrity)

        - **Invocation Property**: prohibits subject at one level of integrity from invoking a subject at a higher level of integrity
        - **Lipner implementation**: by combining Biba and Bell-LaPadula, you get both confidentiality and integrity
        - Biba uses a lattice to control access and is a form of mandatory access control (MAC) model
- **Take-Grant**:
    - Take-grant is a confidentiality-based model that supports four basic operations: take, grant, create, and revoke; it employs a directed graph to dictate how rights can be passed from one subject to another, or from a subject to an object
    - **Take rule**: allows a subject to take rights over an object
    - **Grant rule**: allows a subject to grant rights to an object
    - **Create rule**: allows a subject to create new rights
    - **Remove rule**: allows a subject to remove rights it has
- **Clark-Wilson**:
    - Designed to protect integrity using the access control triplet (subject/program/object)
    - Clark-Wilson has three rules of integrity:
        - well-formed transactions (transactions must follow specific rules)
        - certification rule (processes must be certified as meeting the model's requirements)
        - enforcement rule (the system must enforce the certification rule)
    - A program interface is used to limit what is done by a subject; if the focus of an intermediary program between subject and object is to protect integrity, then it is an implementation of the Clark-Wilson model
    - Uses security labels to grant access to objects via transformation procedures and a restricted interface model
    - The Clark-Wilson Model enforces the concept of separation of duties
    - Three parts of the Clark-Wilson model are: subject, object, and program (or interface)
    - Clark-Wilson components:
        - **Users**: (AKA active agents) the subjects which will access the objects
        - **Transformation Procedures (TPs)**: operations the subject is trying to perform (read, write, modify)
        - **Constrained Data Items (CDIs)**: objects at a higher-level of protection; CDIs can only be manipulated by a TP

        - **Unconstrained Data Items (UDIs)**: can be accessed directly by the subject, and do not need to go through a intermediary like a TP
        - **Integrity Verification Procedures (IVPs)**: a way to audit the TP, checking for internal and external consistency
- **Brewer and Nash Model**:
    - AKA "ethical wall", and "cone of silence"
    - Primarily used to prevent conflicts of interest, by restricting access to information that could create a conflict; created to permit access controls that change dynamically based on a user's previous activity
- **Goguen-Meseguer Model**:
    - An integrity model
    - Foundation of noninterference conceptual theories
- **Sutherland Model**:
    - Focuses on preventing interference in support of integrity; might be used to prevent covert channel attacks
- **Graham-Denning Model**
    - Graham-Denning is primarily an access control model; focused on the secure creation and deletion of both subjects and objects
    - 8 primary protection rules or actions
        - 1-4: securely create/delete a subject/object
        - 5-8: securely provide the read/grant/delete/transfer access right
- **Harrison-Ruzzo-Ullman Model**:
    - Focuses on the assignment of object access rights to subjects as well as the resilience of those assigned rights
    - HRU is an extension of Graham-Denning model
- **Star Model**:
    - Not an official model, but name refers to using asterisks (stars) to dictate whether a person at a specific level of confidentiality is allowed to write data to a lower level of confidentiality
    - Also determines whether a person can read or write to a higher or lower level of confidentiality

3.3 Select controls based upon systems security requirements (OSG-10 Chpt 8)

- Be familiar with the **Common Criteria (CC)** for Information Technology Security Evaluation:
    - Based on ISO/IEC 15408, CC is a subjective security function evaluation tool that uses protection profiles (PPs), security targets (STs), and assigns an Evaluation Assurance level (EAL)
    - The CC provides a standard to evaluate systems, defining various levels of testing and confirmation of systems' security capabilities
    - The number of the level indicates what kind of testing and confirmation has been performed
    - The important concepts:
        - To perform an evaluation, you need to select the **Target of Evaluation (TOE)** (e.g. firewall or an anti-malware app)
        - The evaluation process will look at the **protection profile (PP)**, which is a document that outlines the security needs (customer "I wants"); a vendor might use a specific protection profile for a particular solution
        - The evaluation process will look at the **Security Target (ST)**, specifying the claims of security from the vendor that are built into a TOE (the ST is usually published to customers and partners and available to internal staff)
        - An organization's PP is compared to various STs from the selected vendor's TOEs, and the closest or best match is what the org purchases
        - The evaluation will attempt to gauge the confidence level of a security feature
        - **Security assurance requirements (SARs)**: a description of how the TOE is to be evaluated, based on the development of the solution
        - Key actions during development and testing should be captured
        - An **evaluation assurance level (EAL)**: a numerical rating used to assess the rigor of an evaluation; the scale is EAL 1 (cheap and easy) to EAL7 (expensive and complex):
            - EAL1: functionally tested
            - EAL2: structurally tested
            - EAL3: methodically tested and checked
            - EAL4: methodically designed, tested, and reviewed

        - EAL5: semi-formally designed and tested
        - EAL6: semi-formally verified, designed, and tested
        - EAL7: formally verified, designed, and tested
- **Authorization to Operate (ATO)**: official auth to use specific IT systems to perform tasks/accept identified risks; an ATO assessment is done by an authorizing official (AO); an ATO must be renewed/re-obtained when:
    - A system has a significant security change or security breach
    - the ATO time period expires

3.4 Understand security capabilities of Information Systems (IS) (e.g. memory protection, Trusted Platform Model (TPM), encryption/decryption) (OSG-10 Chpt 8)

- Security capabilities of information systems include memory protection, virtualization, Trusted Platform Module (TPM), encryption/decryption, interfaces, and fault tolerance
- A computing device is likely running multiple apps and services simultaneously, each occupying a segment of memory; the goal of memory protection is to prevent one app or service from impacting another
- There are two primary memory protection methods:
    - **Process isolation**: OS provides separate memory spaces for each process instructions and data, and prevents one process from impacting another
    - **Hardware segmentation**: forces separation via physical hardware controls rather than logical processes; in this type of segmentation, the operating system maps processes to dedicated memory locations
- **Virtualization**: technology used to host one or more operating systems within the memory of a single host, or to run applications that are not compatible with the host OS; the goal is to protect the hypervisor and ensure that compromising one VM doesn't affect others on that host
- **Virtual Software**: software that is deployed in a way that acts as if it is interacting with a full host OS; a virtualized app is isolated from the host OS so it can't make direct/permanent changes to the host OS
- **Reference Monitor Concept (RMC)**: theoretical concept that provides a way to check and control every subject to object access attempt; four properties (**NEAT**): **N**on-bypassable, **E**valuable, **A**lways invoked, **T**amper-proof
- **Security Kernel** is simply an implementation of the reference monitor concept
- **Trusted Platform Module (TPM)**: a cryptographic chip that is sometimes included with a client computer or server; a TPM enhances the capabilities of a computer by offering hardware-based cryptographic operations

        - TPM is a tamper-resistant integrated circuit built into some motherboards that can perform cryptographic operations (including key gen) and protect small amounts of sensitive info, like passwords and cryptographic keys
        - Many security products and encryption solutions require a TPM
        - TPM is both a specification for a cryptoprocessor chip on a motherboard and the general name for implementation of the specification
        - A TPM is an example of a **hardware security module (HSM)**: a cryptoprocessor used to manage and store digital encryption keys, accelerate crypto operations, support faster digital signatures, and improve authentication
- User interface: a constrained UI can be used in an application to restrict what users can do or see based on their privileges
    - e.g. dimming/graying out capabilities for users without the correct privileges
    - An interface is also the method by which two or more systems communicate
- Be aware of the common security capabilities of interfaces:
    - Encryption/decryption: when communications are encrypted, a client and server can communicate without exposing information to the network; when an interface doesn't provide such a capability, use IPsec or another encrypted transport mechanism
    - Signing: used for non-repudiation; in a high-security environment, both encrypt and sign all communications if possible
- **Protection rings**: Ring 0, most privileged (aka Kernel) to Ring 3, least privileged (user apps)
- **Fault tolerance**: capability used to enhance availability; in the event of an attack (e.g. DoS), or system failure, fault tolerance helps keep a system up and running

3.5 Assess and mitigate the vulnerabilities of security architectures, designs and solution elements (OSG-10 Chpts 6,7,9,16,20)

This objective relates to identifying vulnerabilities and corresponding mitigating controls and solutions; the key is understanding the types of vulnerabilities commonly present in different environments, and their mitigation options

- 3.5.1 Client-based systems
    - **Client-based systems**: client computers are the most attacked entry point
    - Compromised client computers can be used to launch other attacks
    - Productivity software and browsers are constant targets
    - Even patched client computers are at risk due to phishing and social engineering vectors
    - Mitigation: run a full suite of security software, including EDR/XDR, anti-virus/malware, and host-based firewall
    - **Mobile Device Management (MDM)**: software and processes to manage, monitor, and secure mobile devices
    - **Mobile Device Deployment Policies**: understand BYOD, CYOD, COPE, and COBO
- 3.5.2 Server-based systems
    - **Type 1 hypervisor**: essentially replaces the host operating system, loading directly onto the hardware
    - **Type 2 hypervisor**: is installed as a standard application on a conventional OS
    - **VM Escape**: occurs when an attacker exploits a vulnerability in the hypervisor to “break out” of their isolated VM and gain unauthorized access to the underlying host system
    - **VM sprawl**: the uncontrolled proliferation of virtual machines (VMs) that results in inefficient resource use, management complexity, and security risks
    - **Data Flow Control**: movement of data between processes, between devices, across a network, or over a communications channel
    - **Virtual Desktop Infrastructure (VDI)**: a technology that hosts and manages desktop operating systems (like Windows or Linux) and applications on a centralized server

    - **System hardening**: a collection of tools, techniques, and best practices used to reduce security vulnerabilities across an entire technology environment
    - Management of data flow seeks to minimize latency/delays, keep traffic confidential (i.e. using encryption), not overload traffic (i.e. load balancer), and can be provided by network devices/applications and services
    - While attackers may initially target client computers, servers are often the goal
    - **Mitigation**: regular patching, deploying hardened server OS images for builds, and use host-based firewalls
- 3.5.3 Database systems
    - Databases often store a company's most sensitive data (e.g. proprietary, CC info, PHI, and PII)
    - **Table**: (AKA relation) the basic structure used to store data in a relational database
    - **Row**: (AKA tuple or record) represents a single, complete, and implicitly structured data item in a table
    - **Column**: (AKA an attribute or field) represents a specific property or characteristic of the entity defined by the table
    - **Cardinality**: refers to the number of rows in a table
    - **Candidate Key**: an attribute, or a minimal set of attributes, that can uniquely identify every row in a table
    - **Primary Key**: the candidate key chosen by the database designer to be the principal unique identifier for the table
    - **Foreign Key**: an attribute or a set of attributes in one table (the referencing or ‘child’ table) that refers to the primary key of another table (the referenced or ‘parent’ table)
    - **Referential integrity**: ensures that values in the foreign key column of the child table must already exist in the primary key column of the parent table
    - **Degree**: refers to the number of columns in a table
    - **ACID model**: set of properties that guarantee the validity of database transactions:
        - **Atomicity**: transactions are all-or-nothing; a transaction must be an atomic unit of work, i.e., all of its data modifications are performed, or none are performed
        - **Consistency**: transactions must leave the database in a consistent state

        - **Isolation**: transactions are processed independently
        - **Durability**: once a transaction is committed, it is permanently recorded
    - Attackers may use inference or aggregation to obtain confidential information
    - **Aggregation attack**: based on math; process where SQL provides a number of functions that combine records from one or more tables to produce potentially useful info
    - **Inference attack**: based on human deduction; involves combining several pieces of nonsensitive info to gain access to that which should be classified at a higher level; inference makes use of the human mind's deductive capacity rather than the raw mathematical ability of database platforms
- 3.5.4 Cryptographic systems
    - Goal of a well-implemented cryptographic system is to make compromise too time-consuming and/or expensive
    - Each component has vulnerabilities:
        - **Kerckhoff's Principle** (AKA Kerckhoff's assumption): a cryptographic system should be secure even if everything about the system, except the key, is public knowledge
        - Software: used to encrypt/decrypt data; can be a standalone app, command-line, built into the OS or called via API; like any software, there are likely bugs/issues, so regular patching is important
        - Keys: dictate how encryption is applied through an algorithm; a key should remain secret, otherwise the security of the encrypted data is at risk
            - **key space**: represents all possible permutations of a key
            - key space best practices:
                - key length is an important consideration; use as long of a key as possible (your goal is to outpace projected increase in cryptanalytic capability during the time the data must be kept safe); longer keys discourage brute-force attacks
                    - a 256-bit key is typically minimum recommendation for symmetric encryption
                    - 2048-bit key typically the minimum for asymmetric

                - always store secret keys securely, and if you must transmit them over a network, do so in a manner that protects them from unauthorized disclosure
                - select the key using an approach that has as much randomness as possible, taking advantage of the entire key space
                - destroy keys securely, when no longer needed
            - always base key length on requirements and sensitivity of the data being handled
        - Algorithms: choose algorithms (or ciphers) with a large key space and a large random **key value** (key value is used by an algorithm for the encryption process)
            - algorithms themselves are not secret; they have extensive public details about history and how they function
- 3.5.5 Industrial Control Systems (ICS)
    - **Industrial control systems (ICS)**: a form of computer-management device that controls industrial processes and machines, also known as operational technology (OT); there are several forms of ICS including distributed control systems (DCS), programmable logic controllers (PLC), and supervisory control and data acquisition (SCADA)
        - recognize that DCS, PLC, and SCADA are types of ICS; and know about how to secure ICS
    - **Supervisory control and data acquisition (SCADA)**: systems used to control physical devices like those in an electrical power plant or factory; SCADA systems are well suited for distributed environments, such as those spanning continents
        - some SCADA systems still rely on legacy or proprietary communications, putting them at risk, especially as attackers gain knowledge of such systems and their vulnerabilities
    - ICS risk mitigations:
        - network segmentation
        - robust change management: document changes, and configurations; only apply vendor-approved patches
        - physical security: controls should include physical access restrictions to components
        - encryption: use secure communication between components
        - system hardening/logging: disable unused services/ports, log and monitor activities

    - **DCS (Distributed Control System)**: a control system usually found within a single factory or plant; uses a network of controllers managed by a central supervisory control level to control local processes
    - **PLC (Programmable Logic Controller)**: a rugged, industrial computer used to automate specific electromechanical processes
- 3.5.6 Cloud-based systems (e.g., Software as a Service (SaaS), Infrastructure as a Service (IaaS), Platform as a Service (PaaS))
    - **Software as a Service (SaaS)**: provides fully functional apps typically accessible via a web browser
    - **Platform as a Service (PaaS)**: provide consumers with a computing platform, including hardware, operating systems, and a runtime environment
    - **Infrastructure as a Service (IaaS)**: provide basic computing resources like servers, storage, and networking
        - note that the cloud service provider providing the least amount of maintenance and security is the IaaS model
    - **Cloud-based systems**: on-demand access to computing resources available from almost anywhere
        - Cloud's primary challenge: resources are outside the org’s direct control, making it more difficult to manage risk
        - Orgs should formally define requirements to store and process data stored in the cloud
        - Focus your efforts on areas that you can control, such as the network entry and exit points (i.e. firewalls and similar security solutions)
        - All sensitive data should be encrypted, both for network communication and data-at-rest
        - Use centralized identity access and management system, with multifactor authentication
        - Customers shouldn’t use encryption controlled by the vendor, eliminating risks to vendor-based insider threats, and supporting destruction using cryptographic erase
    - **Community cloud**: the cloud environment is maintained, used, and paid for as a shared benefit by associated users or organizations; benefits might include collaboration, data exchange, and cost savings compared to private or public clouds
    - Capture diagnostic and security data from cloud-based systems and store in your SIEM system

- Ensure cloud configuration matches or exceeds your on-premise security requirements
- Understand the cloud vendor's security strategy
- Cloud shared responsibility by model:
    - Software as a Service (SaaS):
        - the vendor is responsible for all maintenance of the SaaS services
    - Platform as a Service (PaaS):
        - customers deploy apps that they've created or acquired, manage their apps, and modify config settings on the host
        - the vendor is responsible for maintenance of the host and the underlying cloud infrastructure
    - Infrastructure as a Service (IaaS):
        - IaaS models provide basic computing resources to customers
        - customers install OSs and apps and perform required maintenance
        - the vendor maintains cloud-based infra, ensuring that customers have access to leased systems

- 3.5.7 Distributed systems
    - **Distributed computing environment (DCE)**: a collection of individual systems that work together to support a resource or provide a service
    - DCEs are designed to support communication and coordination among their members in order to achieve a common function, goal, or operation
    - Most DCEs have duplicate or concurrent components, are asynchronous, and allow for fail-soft or independent failure of components
    - DCE is AKA concurrent computing, parallel computing, and distributed computing
    - DCE solutions are implemented as client-server, three-tier, multi-tier, and peer-to-peer
    - Securing distributed systems:
        - In distributed systems, integrity is sometimes a concern because data and software are spread across various systems, often in different locations
        - Client/server model network is AKA a distributed system or distributed architecture
            - security must be addressed everywhere instead of at a single centralized host

        - processing and storage that are distributed on multiple clients and servers, and all must be secured
        - network links must be secured and protected
- 3.5.8 Internet of Things (IoT)
    - **Internet of things (IoT)**: a class of smart devices that are internet-connected in order to provide automation, remote control, or AI processing to appliances or devices
        - An IoT device is almost always separate/distinct hardware used on its own or in conjunction with an existing system
        - IoT security concerns often relate to access and encryption
        - IoT is often not designed with security as a core concept, resulting in security breaches; once an attacker has remote access to the device they may be able to pivot
        - Securing IoT:
            - deploy a distinct network for IoT equipment, kept separate and isolated (known as **three dumb routers**)
            - keep systems patched
            - limit physical and logical access
            - monitor activity
            - implement firewalls and filtering
            - never assume IoT defaults are good enough, evaluate settings and config options, and make changes to optimize security while supporting business functions
            - disable remote management and enable secure communication only (such as over HTTPS)
            - review IoT vendor to understand their history with reported vulnerabilities, response time to vulnerabilities and their overall approach to security
            - not all IoT devices are suitable for enterprise networks
- 3.5.9 Microservices (e.g., application programming interface (API))
    - **Service-oriented Architecture (SOA)**: constructs new apps or functions out of existing but separate and distinct software services, and the resulting app is often new; therefore its security issues are unknown, untested, and unprotected; a derivative of SOA is microservices
    - **Microservices**: a feature of web-based solutions and derivative of SOA, microservices app is put together as a collection of loosely-couples small and independent services; A microservice is simply one element, feature,

capability, business logic, or function of a web app that can be called upon or used by other web apps
        - Microservices are usually small and focused on a single operation, engineered with few dependencies, and based on fast, short-term development cycles (similar to Agile)
        - Each microservice exposes an Application Programming Interface (API) providing communication and interaction with other services; these APIs allow for a modular, flexible, and scalable architecture
        - Securing microservices:
            - use HTTPS only
            - encrypt everything possible and use routine scanning
            - closely aligned with microservices is the concept of shifting left, or addressing security earlier in the SDLC; also integrating it into the CI/CD pipeline
            - consider the software supply chain or dependencies of libraries used, when addressing updates and patching
            - ensure APIs are secure by using appropriate authentication, authorization, and encryption for data exchanges
            - if deployed via containers, ensure appropriate access control, secure images and configurations; ensure the software is updated and patched regularly
- 3.5.10 Containerization
    - **Containerization**: AKA OS virtualization, is based on the concept of eliminating the duplication of OS elements in a virtual machine; instead each app is placed into a container that includes only the actual resources needed to support the app, and the common or shared OS elements are used from the host operating system
        - Containerization is able to provide 10 to 100 x more application density per physical server compared to traditional virtualization
        - Vendors often have security benchmarks and hardening guidelines to follow to enhance container security
        - Securing containers:
            - container challenges include the lack of isolation compared to a traditional infrastructure of physical servers and VMs
            - scan container images to reveal software with vulns
            - secure your registries: use access controls to limit who can publish images, or even access the registry
            - require images to be signed

        - harden container deployment including the OS of the underlying host, using firewalls, and VPC rules, and use limited access accounts
        - reduce the attack surface by minimizing the number of components in each container, and update and scan them frequently
- 3.5.11 Serverless
    - **Serverless architecture** (AKA **function as a service (FaaS)**): cloud computing where code is managed by the customer and the platform (i.e. supporting hardware and software) or servers are managed by the CSP
        - Note that FaaS is a subcategory of PaaS
        - Applications developed on serverless architecture are similar to microservices, and each function is created to operate independently and autonomously
        - A serverless model, as in other CSP models, is a shared security model, and your org and the CSP share security responsibility
- 3.5.12 Embedded systems
    - **Embedded systems**: any form of computing component added to an existing mechanical or electrical system for the purpose of providing automation, remote control, and/or monitoring; usually including a limited set of specific functions
        - Embedded systems can be a security risk because they are generally static, with admins having no way to update or address security vulns (or vendors are slow to patch)
        - Embedded systems focus on minimizing cost and extraneous features
        - Embedded systems are often in control of/associated with physical systems, and can have real-world impact
        - Securing embedded systems:
            - embedded systems should be isolated from the internet, and from a private production network to minimize exposure to remote exploitation, remote control, and malware
            - use secure boot feature and physically protecting the hardware
- 3.5.13 High-Performance Computing systems
    - **High-performance computing (HPC)** systems: platforms designed to perform complex calculations/data manipulation at extremely high

speeds (e.g. super computers or MPP (Massively Parallel Processing)); often used by large orgs, universities, or gov agencies
        - An HPC solution is composed of three main elements:
            - compute resources
            - network capabilities
            - storage capacity
        - HPCs often implement real-time OS (RTOS)
        - HPC systems are often rented, leased or shared, which can limit the effectiveness of firewalls and invalidate air gap solutions
        - Securing HPC systems:
            - deploy head nodes and route all outside traffic through them, isolating parts of a system
            - "fingerprint" HPC systems to understand use, and detect anomalous behavior
- 3.5.14 Edge computing systems
    - **Edge computing**: philosophy of network design where data and compute resources are located as close as possible, at or near the network edge, to optimize bandwidth use while minimizing latency; intelligence and processing are contained within each device, and each device can process its own data locally
    - **Fog Computing**: a larger, more distributed extension of cloud computing that sits as an intermediate layer between the edge and the central Cloud
    - One of the primary benefits of edge and fog computing from a security perspective is that they reduce the amount of data flowing to the cloud, which improves data privacy and reduces the risk of data breaches
    - Edge and Fog security recommendations:
        - network segmentation
        - data encryption and strong authentication
        - regular patching
        - visibility, control, and correlation requires a Zero Trust access-based approach to address security on the LAN edge, WAN edge and cloud edge, as well as network management
        - devices on your network, no matter where they reside, need to be configured, managed, and patched using a consistent policy and enforcement strategy
        - use intelligence from side-channel signals that can pick up hardware trojans and malicious firmware
        - attend to physical security

        - deploy IDS on the network side to monitor for malicious traffic
        - in many scenarios, you are an edge customer, and likely will need to rely on a vendor for some of the security and vulnerability remediation
- 3.5.15 Virtualized systems
    - **Virtualized systems**: used to host one or more OSs within the memory of a single host computer, or to run apps not compatible with the host OS
        - Securing virtualized systems:
            - the primary component in virtualization is a hypervisor which manages the VMs, virtual data storage, virtual network components
            - the hypervisor represents an additional attack surface
            - in virtualized environments, you need to protect both the VMs and the physical infrastructure/hypervisor
            - hypervisor admin accounts/credentials and service accounts are targets because they often provide access to VMs and their data; these accounts should be protected
            - virtual hosts should be hardened; to protect the host, avoid using it for anything other than hosting virtualized elements
            - virtualized systems should be security tested via vuln assessment and penetration testing
            - virtualization doesn't lessen the security management requirements of an OS, patch management is still required
            - be aware of VM Sprawl and Shadow IT
            - VM escape minimization:
                - keep highly sensitive systems and data on separate physical machines
                - keep all hypervisor software current with vendor-released patches
                - monitor attack, exposure and abuse indexes for new threats to virtual machines (which might be better protected); often, virtualization administrators have access to all virtuals

3.6 Select and determine cryptographic solutions (OSG-10 Chpts 6,7)

- 3.6.1 Cryptographic lifecycle (e.g., keys management, algorithm selection)
    - **Cryptographic Life Cycle**: the complete process of managing cryptographic keys, algorithms, and components from initial system creation to final destruction
    - Keep **Moore's Law** in mind (processing capabilities of state-of-the-art microprocessors double approximately every 18 to 24 months), and have appropriate governance controls in place to ensure that algorithms, protocols, and key lengths selected are sufficient to preserve the integrity of the cryptosystems for as long as necessary -- to keep secret information safe
    - Specify the cryptographic algorithms (such as AES, 3DES, and RSA) acceptable for use in an organization
    - Identify the acceptable key lengths for use with each algorithm based on the sensitivity of the info transmitted
    - Enumerate the secure transaction protocols (e.g. TLS) that may be used
    - As computing power goes up, the strength of cryptographic algorithms goes down; keep in mind the effective life of a certificate or cert template, and of cryptographic systems
    - TLS uses an ephemeral symmetric session key between server and client, exchanged using asymmetric cryptography; note that all content is protected using symmetric cryptography
    - Beyond brute force, consider things like the discovery of a bug or an issue with an algorithm or system
    - NIST defines the following terms that are commonly used to describe algorithms and key lengths:
        - approved (an algorithm that is specified as a NIST or FIPS recommendation)
        - acceptable (algorithm + key length is safe today)
        - deprecated (algorithm and key length is OK to use, but brings some risk)
        - restricted (use of the algorithm and/or key length is deprecated and should be avoided)
        - legacy (the algorithm and/or key length is outdated and should be avoided when possible)
        - disallowed (algorithm and/or key length is no longer allowed for the indicated use)

- 3.6.2 Cryptographic methods (e.g., symmetric, asymmetric, elliptic curves, quantum)
    - **Confusion**: the principle of making the relationship between the key and the ciphertext as complex as possible
    - **Diffusion**: the principle of spreading the influence of a single plaintext bit over as much of the ciphertext as possible
    - **Symmetric** encryption: uses the same key for encryption and decryption
        - symmetric encryption uses a shared secret key available to all users of the cryptosystem
        - symmetric encryption is faster than asymmetric encryption because smaller keys can be used for the same level of protection
        - downside is that users or systems must find a way to securely share the key and hope the key is used only for the specified communication
        - symmetric-key encryption can use either stream ciphers or block ciphers
        - primarily employed to perform bulk encryption and provides only for the security service of confidentiality
        - "same" is a synonym for symmetric
        - "different" is a synonym for asymmetric
        - **total number of keys** required to completely connect n parties using symmetric cryptography is given by this formula:
            - **(n(n - 1)) / 2**
        - symmetric cryptosystems operate in several discrete modes:
            - **Electronic Code Book (ECB) mode**: the simplest and weakest of the modes; each block of plaintext is encrypted separately, but they are encrypted in the same way
                - advantages: fast, blocks can be processed simultaneously
                - disadvantages: any plaintext duplication would produce the same ciphertext
            - **Cipher Block Chaining (CBC) mode**: a block cipher mode of operation that encrypts plaintext by using an operation called XOR (exclusive-OR); XORing a block with the previous ciphertext block is known as "chaining"; this means that the decryption of a block of ciphertext depends on all the preceding ciphertext blocks; CBC uses an Initialization Vector or IV, which is a random value or nonce shared between sender and receiver

    - advantages: CBC uses the previous ciphertext block to encrypt the next plaintext block, making it harder to deconstruct; XORing process prevents identical plaintext from producing identical ciphertext; a single bit error in a ciphertext block affects the decryption of that block and the next, making it harder for attackers to exploit errors
    - disadvantages: the blocks must be processed in order, not simultaneously (so it's slower); CBC is also vulnerable to POODLE and GOLDENDOODLE attacks
- **Cipher Feedback (CFB) mode**: streaming version of CBC; similar to CBC, it uses an IV and the cipher from the previous block so errors can propagate; the main difference is that with CFB, the cipher from the previous block is encrypted first, then XORed with the current block
    - advantages: CFB is considered to be faster than CBC even though it’s also sequential
    - disadvantages: if there’s an error in one block, it can carry over into the next block
- **Output Feedback (OFB) mode**: OFB turns a block cipher into a synchronous stream cipher; based on an IV and the key, it generates keystream blocks which are then simply XORed with the plaintext data; as with CFB, the encryption and decryption processes are identical, and no padding is required
    - advantages: OFB mode doesn't use chaining, so errors do not propagate; doesn't need a unique nonce for each message, which can simplify the management and generation of nonces; it's resistant to replay attacks
    - disadvantages: no data integrity protection, vulnerability to IV management issues, and lacks parallelization capabilities due to the dependence of the keystream on previous blocks
- **Counter (CTR) mode**: key feature is that you can parallelize encryption and decryption, and it doesn’t require chaining; it uses a counter function to generate a nonce value for each block’s encryption; the nonce number (aka

the counter) gets encrypted and then XORed with the plaintext to generate ciphertext; the resulting ciphertext should also always be unique
        - advantage: CTR mode is fast, and considered to be secure
        - disadvantage: lacks integrity, so we need to use hashing
    - **Galois/Counter (GCM) mode**: combines counter mode (CTR) with Galois authentication; we can not only encrypt data, but we can authenticate where the data came from (providing both data integrity and confidentiality); includes authentication data, and uses hashing as starting values
        - advantages: extremely fast, GCM is recognized by NIST and used in the IEEE 802.1AE standard
        - disadvantages: most stated disadvantages seem to be around implementation burdens
    - **Counter with Cipher Block Chaining Message Authentication Code (CCM) mode**: uses counter mode so there is no error propagation; there is no duplication, but uses chaining, so cannot run in parallel; MAC or message authentication code: provides authentication and integrity
        - advantages: no error propagation, provides authentication and integrity
        - disadvantages: cannot be run in parallel
    - Examples of symmetric algorithms: Twofish, Serpent, AES (Rijndael), Camellia, Salsa20, ChaCha20, Blowfish, CAST5, Kuznyechik, RC4/5/6, DES, 3DES, Skipjack, Safer, and IDEA (remember that DES has been considered insecure for decades, and NIST formally deprecated 3DES in 2023)
- **Asymmetric** encryption: process that uses different keys for encryption and decryption, and in which the decryption key is computationally not possible to determine given the encryption key itself
    - Asymmetric (AKA public key, since one key of a pair is available to anybody) algorithms provide convenient key exchange mechanisms and are scalable to very large numbers of users (addressing the two most significant challenges for users of symmetric cryptosystems)

    - Asymmetric cryptosystems avoid the challenge of sharing the same secret key between users, by using pairs of public and private keys to allow secure communication without the overhead of complex key distribution
    - **Public key**: one part of the matching key pair, which can be shared or published
    - Besides the public key, there is a private key that should remain private and protected
    - Private key secrecy and integrity of an asymmetric encryption process are entirely dependent upon protecting the value of the private key
    - While asymmetric encryption is slower, it is best suited for sharing between two or more parties
    - Asymmetric encryption provides confidentiality, authentication and non-repudiation
    - Asymmetric key use:
        - to encrypt a message: use the recipient's public key
        - to decrypt a message: use your own private key
    - Most common asymmetric cryptosystems in use today:
        - Rivest-Shamir-Adleman (RSA): depends on factoring the product of prime numbers
        - Diffie-Hellman: depends on modular arithmetic
        - ElGamal: extension of Diffie-Hellman that depends on modular arithmetic
        - Elliptic Curve Cryptography (ECC): elliptic curve algorithm depends on the elliptic curve discrete logarithm problem and provides more security than other algorithms when both are used with keys of the same length
- **Hash function**: a one-way mathematical function that takes an input (or message) of any size and produces a fixed-size string of characters called a hash value, hash code, or digest
- **Quantum cryptography**: quantum computing is a newer and advanced type of technology with the promise of future power to enhance fields like AI, encryption, medicine, and science; classical computing uses electrical or optical on/off impulses representing 0s and 1s, and quantum computing's power lies in harnessing quantum mechanical principles allowing qubits (quantum bits) to represent both 0 and 1 simultaneously and multidimensionally

        - **Quantum supremacy**: the potential for quantum computing to easily resolve hard problems (e.g. factoring large integers and solving discrete logarithms) rendering algorithms like RSA and Diffie-Hellman insecure
        - **Post-quantum cryptography (PQC)**: it's difficult to estimate when quantum computing will render current encryption methods irrelevant (or even possibly if it already has); Harvest Now, Decrypt later (HDNL) is the threat of adversarial interception and storing of encrypted data, with the intent of using quantum computers to decrypt it in the future
            - security professionals need to consider the useful time period of currently encrypted data, and take steps to begin incorporating PQC-safe algorithms
    - Check out Practical Cryptography for Developers for a deeper dive
- 3.6.3 Public Key Infrastructure (PKI) (e.g., quantum key distribution)
    - **Public Key Infrastructure (PKI)**: hierarchy of trust relationships permitting the combination of asymmetric and symmetric cryptography along with hashing and digital certificates (giving us hybrid cryptography)
        - A PKI issues certificates to computing devices and users, enabling them to apply cryptography (e.g., to send encrypted email messages, encrypt websites or use IPsec to encrypt data communications)
        - Many vendors provide PKI services; you can run a PKI privately and solely for your own org, you can acquire certificates from a trusted third-party provider, or you can do both (which is common)
        - A PKI is made up of:
            - **certification authorities (CAs)**: servers that provide one or more PKI functions, such as providing policies or issuing certificates
            - **registration authority (RA)**: entity that verifies the identity of the user or device requesting a certificate
            - **digital certificates**: The core of PKI, these electronic credentials bind a user’s identity to their public key
            - policies and procedures: such as how the PKI is secured
            - templates: a predefined configuration for specific uses, such as a web server template

        - CAs generate digital certificates containing the public keys of system users; users then distribute these certificates to people with whom they want to communicate; certificate recipients verify a certificate using the CA's pubic key
- **Quantum key distribution (QKD)**: while traditional methods of securely distributing symmetric keys are via either using an out-of-band channel (some alternate secure communication method), or hybrid method (e.g. asymmetric methods like via Diffie-Hellman), a potentially new secure communication method uses quantum mechanics to exchange encryption keys between two parties, where, assuming Heisenberg's Uncertainty Principle, measuring a quantum state inherently disrupts it, eavesdropping on a QKD would be detectable ("unconditional security")
    - There are other components and concepts you should know for the exam:
        - A PKI can have multiple tiers:
            - single tier means you have one or more servers that perform all the functions of a PKI
            - two tiers means there is an offline root CA (a server that issues certificates to the issuing CAs but remains offline most of the time) in one tier, and issuing CAs (the servers that issue certificates to computing devices and users) in the other tier
                - servers in the second tier are often referred to as intermediate CAs or subordinate CAs
            - three tier means you can have CAs that are responsible only for issuing policies (and they represent the second tier in a three-tier hierarchy)
                - in such a scenario, the policy CAs should also remain offline and be brought online only as needed
            - Generally, the more tiers, the more security (but proper configuration is critical)
                - the more tiers you have, the more complex and costly the PKI is to build and maintain
            - A PKI should have a certificate policy and a certificate practice statement (CSP)

        - certificate policy: documents how your org handles items like requestor identities, the uses of certificates and storage of private keys
        - CSP: documents the security configuration of your PKI and is usually available to the public
    - Besides issuing certificates, a PKI has other duties:
        - a PKI needs to be able to provide certificate revocation information to clients
        - if an administrator revokes a certificate that has been issued, clients must be able to get that info from your PKI
        - storage of private keys and info about issued certificates (can be stored in a database or a directory)
    - PKI uses LDAP when integrating digital certs into transmissions
- Key management
    - **Key management**: the full set of processes and procedures for managing cryptographic keys throughout their lifecycle, including generating, distributing, storing, using, rotating, revoking, archiving, and ultimately destroying keys in a secure and controlled manner
    - **Key management practices**: include safeguards surrounding the creation, distribution, storage, destruction, recovery, and escrow of secret keys
        - Cryptography can be used as a security mechanism to provide confidentiality, integrity, and availability only if keys are not compromised
        - Three main methods are used to exchange secret keys:
            - offline distribution
            - public key encryption, and
            - the Diffie-Hellman key exchange algorithm
        - Key management can be difficult with symmetric encryption but is much simpler with asymmetric encryption
        - There are several tasks related to key management:
            - Key creation
            - **Key distribution**: the process of sending a key to a user or system; it must be secure and it must be stored in a secure way on the computing device

        - keys are stored before and after distribution; when distributed to a user, it can't hang out on a user's desktop
    - Keys shouldn't be in cleartext outside the cryptography device
    - Key distribution and maintenance should be automated (and hidden from the user)
    - Keys should be backed up!
    - **Key escrow**: process or entity that can recover lost or corrupted cryptographic keys
        - **multiparty key recovery**: when two or more entities are required to reconstruct or recover a key
        - **m of n control**: you designate a group of (n) people as recovery agents, but only need subset (m) of them for key recovery
        - **split custody**: enables two or more people to share access to a key (e.g. for example, two people each hold half the password to the key)
    - Key rotation: rotate keys (retire old keys, implement new) to reduce the risks of a compromised key having access
    - **Cryptographic erase**: methods that permanently remove the cryptographic keys
    - Key states:
        - suspension: temporary hold
        - revocation: permanently revoked
        - expiration
        - destruction
    - See NIST 800-57, Part 1
- Digital signatures
    - **Digital signatures**: provide proof that a message originated from a particular user of a cryptosystem, and ensures that the message was not modified while in transit between two parties
        - Digital signatures rely on a combination of two major concepts — public key cryptography, and hashing functions
        - Digitally signed messages assure the recipient that the message truly came from the claimed sender, enforcing nonrepudiation
        - Digitally signed messages assure the recipient that the message was not altered while in transit; protecting against both malicious

modification (third party altering message meaning), and unintentional modification (faults in the communication process)
        - Digital signature process does not provide confidentiality in and of itself (only ensures integrity, authentication, and nonrepudiation)
        - To digitally sign a message, first use a hashing function to generate a message digest; then encrypt the digest with your private key
        - To verify a digital signature, decrypt the signature with the sender's public key and compare the message digest to the one you generate yourself: if they match, the message is authentic
    - FIPS 186-5 Digital Signature Standard (DSS) - specifies three techniques for the generation and verification of digital signatures that can be used for the protection of data:
        - the Rivest-Shamir-Adleman Algorithm (RSA)
        - the Elliptic Curve Digital Signature Algorithm (ECDSA), and
        - the Edwards-Curve Digital Signature Algorithm (EdDSA)
        - NOTE: the Digital Signature Algorithm (DSA) is now to be used only for verifying existing signatures

3.7 Understand methods of cryptanalytic attacks (OSG-10 Chpts 7,14,21)

- 3.7.1 Brute force
    - **Brute force**: an attack that attempts every possible valid combination for a key or password
        - they involve using massive amounts of processing power to methodically guess the key used to secure cryptographic communications
    - **Rainbow Tables**: large, precomputed databases that store chains of plaintext passwords and their corresponding cryptographic hash values, allowing an attacker to quickly look up a hash and retrieve the original plaintext password without having to compute hashes in real-time
    - **Dictionary Attack**: a password-cracking method in which the attacker systematically tries every word in a dictionary or word list
    - **Hybrid Attack**: is a password-cracking method that combines elements of both dictionary and brute-force attacks; better success rate compared to a pure dictionary attack, and faster than a full brute-force method
    - Brute force countermeasures:
        - strong password policies

        - password hashing and salting
        - key stretching
- 3.7.2 Ciphertext only
    - **Ciphertext only**: an attack where you only have the encrypted ciphertext message at your disposal (not the plaintext)
        - if you have enough ciphertext samples, the idea is that you can decrypt the target ciphertext based on the samples
        - frequency analysis is a technique that is helpful against simple ciphers (see below)
    - Countermeasures:
        - use a strong, well-vetted encryption algorithm and proper key management
- 3.7.3 Known plaintext
    - **Known plaintext**: in this attack, the attacker has a copy of the encrypted message along with the plaintext message used to generate the ciphertext (the copy); this knowledge greatly assists the attacker in breaking weaker codes; the goal is to use the plaintext and associated ciphertext to deduce the encryption key
    - **Linear cryptanalysis**: a known plaintext attack, in which the attacker studies probabilistic linear relations referred to as linear approximations among parity bits of the plaintext, the Ciphertext and the hidden key; considered most effective technique in exploiting statistical properties of ciphertext to find correlations between plaintext and ciphertext
    - **Meet-in-the-middle** typically a known-plaintext attack that works by performing two simultaneous, smaller brute-force searches with different keys: both encryption of the plaintext and decryption of the ciphertext simultaneously to identify the encryption key; 2DES is vulnerable to this attack
    - **Chosen-plaintext attack (CPA)**: an attack model for cryptanalysis which presumes that the attacker can obtain the ciphertexts for arbitrary plaintexts, with the goal to gain information that reduces the security of the encryption scheme; a CPA is more powerful than a known plaintext attack; however a chosen-plaintext is less powerful than a chosen ciphertext
- 3.7.4 Frequency analysis
    - **Frequency analysis**: an attack where the characteristics of a language are used to defeat substitution ciphers

        - for example in English, the letter "E" is the most common, so the most common letter in an encrypted ciphertext could be a substitution for "E"
        - other examples might include letters that appear twice in sequence, as well as the most common words used in a language
    - Countermeasures: modern cryptographic systems are designed specifically to defeat this attack by ensuring that the ciphertext appears statistically random
- 3.7.5 Chosen ciphertext
    - **Chosen ciphertext**: in a chosen ciphertext attack, the attacker has access to one or more plaintexts of arbitrary ciphertexts; i.e. the attacker has the ability to decrypt chosen portions of the ciphertext message, and use the decrypted portion to discover the key
    - **Differential cryptanalysis**: a type of chosen plaintext attack, and a general form of cryptanalysis applicable primarily to block ciphers, but also to stream ciphers and cryptographic hash functions; it is the study of how differences in information input can affect the resultant difference at the output; advanced methods such as differential cryptanalysis are types of chosen plaintext attacks
        - as an example, an attacker may try to get the receiver to decrypt modified ciphertext, looking for that modification to cause a predictable change to the plaintext
    - Countermeasures: use authenticated encryption modes such as GCM (Galois/Counter Mode), or CCM (Counter with Cipher Block Chaining-Message Authentication Code)
- 3.7.6 Implementation attacks
    - **Implementation attack**: attempts to exploit weaknesses in the implementation of a cryptography system
        - focuses on exploiting the software code, not just errors or flaws but the methodology employed to program the encryption system
        - in this type of attack, attackers look for weaknesses in the implementation, such as a software bug or outdated firmware
- 3.7.7 Side-channel
    - **Side-channel**: these attacks seek to use the way computer systems generate characteristic footprints of activity, such as changes in processor utilization, power consumption, or electromagnetic radiation to monitor system activity and retrieve information that is actively being encrypted

        - similar to an implementation attack, side-channel attacks look for weaknesses outside of the core cryptography functions themselves
        - a side-channel attack could target a computer's CPU, or attempt to gain key information about the environment during encryption or decryption by looking for electromagnetic emissions or the amount of execution time required during decryption
        - side-channel characteristics information are often combined together to try to break down the cryptography
        - timing attack is an example
        - the US government and NATO have long been aware of this vulnerability (referring to the associated security standards as TEMPEST)
    - Countermeasures:
        - physical device/cable shielding
        - use TEMPEST-rated equipment
        - introduce random noise into signals (signal obfuscation)
        - maintain physical security
- 3.7.8 Fault-Injection
    - **Fault-Injection**: the attacker attempts to compromise the integrity of a cryptographic device by causing some type of external fault
        - for example, using high-voltage electricity, high or low temperature, or other factors to cause a malfunction that undermines the security of the device
    - Countermeasures: physical security measures, hardware-based encryption, error detection, audits and testing, and robust software/hardware design
- 3.7.9 Timing
    - **Timing**: timing attacks are an example of a side-channel attack where the attacker measures precisely how long cryptographic operations take to complete, gaining information about the cryptographic process that may be used to undermine its security
    - Countermeasures: ensure all sensitive operations execute in a constant amount of time, or conversely, add random delays to hide actual timing, regardless of the secret data being processed
- 3.7.10 Man-in-the-middle (MITM)
    - **Man-in-the-middle (MITM) (AKA on-path, or Adversary-in-the-middle(AitM))**: in this attack a malicious individual sits between two

communicating parties and intercepts all communications (including the setup of the cryptographic session)
        - attacker responds to the originator's initialization requests and sets up a secure session with the originator
        - attacker then establishes a second secure session with the intended recipient using a different key and posing as the originator
        - attacker can then "sit in the middle" of the communication and read all traffic as it passes between the two parties
    - Countermeasures: use a combination of strong protocols, vigilant practices, and robust authentication such as strong encryption
- 3.7.11 Pass the hash
    - **Pass the hash (PtH)**: a technique where an attacker captures a password hash (as opposed to the password characters) and then simply passes it through for authentication and potentially lateral access to other networked systems
        - the threat actor doesn’t need to decrypt the hash to obtain a plain text password
        - PtH attacks exploit the authentication protocol, as the password's hash remains static for every session until the password is rotated
        - attackers commonly obtain hashes by scraping a system's active memory and other techniques
    - PtH attacks typically exploit NTLM vulns, but attackers also use similar attacks against other protocols, including Kerberos
    - Countermeasures: defense-in-depth strategy that focuses on preventing initial access, securing credentials, and restricting lateral movement
- 3.7.12 Kerberos exploitation
    - **Overpass the Hash**: alternative to the PtH attack, used when NTLM is disabled on the network (AKA pass the key)
    - **Pass the Ticket**: in this attack, attackers attempt to harvest tickets held in the lsass.exe process
    - **Silver Ticket**: a silver ticket uses the captured NTLM hash of a service account to create a ticket-granting service (TGS) ticket (the silver ticket grants the attacker all the privileges granted to the service account)
    - **Golden Ticket**: if an attacker obtains the hash of the Kerberos service account (KRBTGT), they can create tickets at will within Active Directory (this provides so much power it is referred to as having a golden ticket)

        - **Kerberos Brute-Force**: attackers use the Python script kerbrute.py on Linux, and Rubeus on Windows systems; tools can guess usernames and passwords
        - **ASREPRoast**: ASREPRoast identifies users that don't have Kerberos preauthentication enabled
        - **Kerberoasting**: kerberoasting collects encrypted ticket-granting service (TGS) tickets
    - 3.7.13 Ransomware
        - **Ransomware**: a type of malware that weaponizes cryptography
            - using many of the same techniques as other types of malware, ransomware generates an encryption key, and encrypts critical files
            - this encryption renders the data inaccessible to the authorized user or anyone else other than the malware author
            - often threatening to publicly release sensitive data if ransom is not paid
            - seek legal advice prior to engaging with ransomware authors
        - Countermeasures:
            - regular, offline backups
            - AV/EDR/XDR
            - employee training
            - patch management
            - least privilege

3.8 Apply security principles to site and facility design (OSG-10 Chpt 10)

- **Secure facility plan**: outlines the security needs of your org and emphasizes methods or mechanisms to employ to provide security, developed through risk assessment and critical path analysis
    - **critical path analysis (CPA)**: a systematic effort to identify relationships between mission-critical apps, processes, and operations and all the necessary supporting components
    - During CPA, evaluate potential **technology convergence**: the tendency for various technologies, solutions, utilities, and systems to evolve and merge over time, which can result in a single point of failure and a more valuable target
    - A secure facility plan is based on a layered defense model

        - **Site selection**: the evaluation of potential geographic locations for a new facility looking for an optimal location based on a set of criteria to determine
        - Site selection should take into account cost, location, and size (but security should always take precedence), that the building can withstand local extreme weather events, vulnerable entry points, and exterior objects that could conceal break-in
            - Key elements of site selection:
                - inherent risk (e.g. frequency of natural disasters)
                - visibility (e.g. proximity to other buildings/businesses that attract many visitors could affect your site)
                - composition of the surrounding area (e.g. the site's physical characteristics such as perimeter, and standoff-zone)
                - area accessibility (access to fundamental resources such as water, utilities, fuel, medical, fire, and police)
- Facility Design:
    - The top priority of security should always be the protection of the life and safety of personnel
    - In the US, follow the guidelines and requirements from Occupational Safety and Health Administration (OSHA), and Environmental Protection Agency (EPA)
    - A key element in designing a facility for construction is understanding the level required by your org and planning for it before beginning construction
    - **Crime Prevention Through Environmental Design (CPTED)**: a well-established school of thought on "secure architecture" - an architectural approach to building and space design that emphasizes passive features to reduce the likelihood of criminal activity
        - core principle of CPTED is that the design of the physical environment can be managed/manipulated, and crafted with intention in order to create behavioral effects or changes in people present in those areas that result in reduction of crime as well as a reduction of the fear of crime
        - CPTED stresses three main principles:
            - **natural access control**: the subtle guidance of those entering and leaving a building
                - make the entrance point obvious
                - create internal security zones

        - areas of the same access level should be open, but restricted/closed areas should seem more difficult to access
    - **natural surveillance**: any means to make criminals feel uneasy through increased opportunities to be observed
        - walkways/stairways are open, open areas around entrances
        - areas should be well lit
    - **natural territorial reinforcement**: attempt to make the area feel like an inclusive, caring community
- Overall goal is to deter unauthorized people from gaining access to a location (or a secure portion), prevent unauthorized personnel from hiding inside or around the location, and prevent unauthorized from committing crime
- There are several smaller activities tied to site and facility design, such as upkeep and maintenance: if property is run down or appears to be in disrepair, it gives attackers the impression that they can act with impunity on the property

3.9 Design site and facility security controls (OSG-10 Chpt 10)

- Note that although the topics in this section cover mostly interior spaces, physical security is applicable to both interior and exterior of a facility
- 3.9.1 Wiring closets/intermediate distribution frame
    - **Wiring closets/intermediate distribution frame (IDF)**: A wiring closet or IDF is typically the smallest room that holds IT hardware
        - wiring closet is AKA premises wire distribution room, main distribution frame (MDF), intermediate distribution frame (IDF), and telecommunications room, and it is referred to as an IDF in (ISC)^2 CISSP objective 3.9.1
        - where networking cables for the building or a floor are connected to equipment (e.g. patch panels, switches, routers, LAN extenders etc)
        - usually includes telephony and network devices, alarm systems, circuit breaker panels, punch-down blocks, WAPs, video/security
        - may include a small number of servers
        - access to the wiring closest/IDF should be restricted to authorized personnel responsible for managing the IT hardware

        - use door access control (i.e. electronic badge system or electronic combination lock)
        - from a layout perspective, wiring closets should be accessible only in private areas of the building interiors; people must pass through a visitor center and a controlled doorway prior to be able to enter a wiring closet
    - Applicable controls:
        - physical locks and access control (e.g. badge readers or other electronic controls)
        - restricted access
        - surveillance and logging
        - environmental monitoring
- 3.9.2 Server rooms/data centers
    - **Server rooms/data centers**: server rooms, data centers, communication rooms, server vaults, and IT closets are enclosed, restricted, and protected rooms where mission critical servers and networks are housed
        - a server room is a bigger version of a wiring closet, much smaller than a data center
        - a server room typically houses network equipment, backup infrastructure and servers (more archaic versions include telephony equipment)
        - server rooms should be designed to support optimal operation of IT infrastructure and to block unauthorized human access or intervention
        - server rooms should be located at the core of the building (avoid ground floor, top floor, or in the basement)
        - server rooms should have a single entrance (and an emergency exit)
        - server room should block unauthorized access, and entries and exits should be logged
        - datacenters are usually more protected than server rooms, and can include guards and mantraps
        - datacenters can be single-tenant or multi-tenant
    - Applicable controls:
        - physical controls
        - technical controls (CCTV, environmental monitoring etc)
        - admin controls (least privilege, access log reviews and audits)
- 3.9.3 Media storage facilities

        - **Media storage facilities**: often store backup tapes/disks, blank, reusable and other media, and should be protected just like a server room
            - depending on requirements a cabinet or safe could suffice
            - new blank media, and media that is reused (e.g. thumb drives, flash memory cards, portable hard drives) should be protected against theft and data remnant recovery
            - concerns include theft, corruption, data remnant recovery
            - other recommendations:
                - employ a media librarian or custodian
                - use check-in/check-out process for media tracking
                - run a secure drive sanitization or "zeroization" when media is returned
            - note: a safe is a movable secured container that's not integrated into a building's construction; a vault is a permanent safe integrated into construction
- 3.9.4 Evidence storage
    - **Evidence storage**: as cybercrime events continue to increase, it is import to retain logs, audit trails, drive images, VM snapshots and other records of digital events; the evidence storage exists to preserve chain of custody
        - a key part of incident response is to gather evidence to perform root cause analysis
        - an evidence storage room should be protected like a server room or media storage facility
        - an evidence storage room can contain physical evidence (such as a smartphone) or digital evidence (such as a database)
        - protections should include dedicated/isolated storage facilities, offline storage, activity tracking, hash management, access restrictions, and encryption
- 3.9.5 Restricted and work area security
    - **Restricted and work area security**: covers the design and configuration of internal security, including work and visitor areas
        - includes areas that contain assets of higher value/importance which should have more restricted access
        - restricted work areas are used for sensitive operations, such as network/security ops
        - protection should be similar to a server room, but video surveillance is typically limited to entry and exit points
- 3.9.6 Utilities and heating, ventilation, and air conditioning (HVAC)

    - Power management in ascending order: surge protectors, power/power-line conditioner, uninterruptible power supply (UPS), generators
    - Types of UPS:
        - double conversion: functions by taking power from the wall outlet, storing it in a battery, pulling power out of the battery and feeding that power to the device/devices
        - line-interactive: has a surge protector, battery charger/inverter and voltage regulator positioned between the grid power source and the equipment (battery is not in line under normal conditions)
    - Commercial power problem types:
        - **fault**: momentary loss of power
        - **blackout**: usually sustained and complete loss of power
        - **sag**: (AKA dip) momentary low voltage
        - **brownout**: prolonged low voltage
        - **spike**: momentary high voltage
        - **surge**: short-duration high voltage
        - **inrush**: initial surge of power associated with connecting to a power source
    - Think through types of physical controls for HVAC:
        - restrict duct space continuity to controlled areas
        - use separate and redundant HVAC systems for computer equipment
        - rooms containing primarily computers should have:
            - temps kept at 59 to 89.6 deg Fahrenheit (15 to 32 deg Celsius)
            - humidity should be maintained between 20 and 80 percent
            - note ASHRAE (American Society of Heating, Refrigerating and Air-Conditioning Engineers) provides guidelines, which are the industry standard for data center environmental conditions
        - note that even on nonstatic carpeting, if the env has low humidity, a 20k-volt static discharge is possible; and even minimal levels of static electricity can destroy electronic equipment
    - Datacenter:
        - should be on different power circuits from occupied areas
        - common to use a backup generator
- 3.9.7 Environmental issues (e.g., natural disasters, man-made)
    - Environmental monitoring is the process of measuring and evaluating the quality of the environment within a given structure (e.g. temperature,

humidity, dust, smoke), using things like chemical, biological, radiological, and microbiological detectors
    - Environmental issues include fire, earthquakes, power outages, tornados and wind
    - Secondary facilities should be located far enough away from the primary to ensure they won't be damaged by the same event
    - Water leakage and flooding should be addressed in your environmental safety policy and procedures; water and electricity together is sure to cause damage; locate server rooms and critical equipment away from any water source or transport pipes
    - If water-based sprinklers are used for fire suppression, damage to electronic equipment is likely; automate the shutoff of electricity prior to sprinkler trigger
    - Note that Halon starves a fire of oxygen by disrupting the chemical reaction of combustion, but degrades into toxic gases at 900 degrees Fahrenheit, and is not environmentally friendly
- 3.9.8 Fire prevention, detection, and suppression
    - Protecting personnel from harm should always be the most important goal of any security or protection system!
    - In addition to protecting people, fire detection and suppression is designed to keep asset damage caused by fire, smoke, heat, and suppression materials to a minimum
    - **Fire triangle**: three triangle corners represent fuel, heat, and oxygen; the center of the triangle represents the chemical reaction among these three elements
        - if you can remove any one of the three items from the fire triangle, the fire can be extinguished
    - Fire suppression mediums:
        - water suppresses temperature
        - soda acid and other dry powders suppress the fuel supply
        - carbon dioxide (CO2) suppresses the oxygen supply
        - halon substitutes and other nonflammable gases interfere with the chemistry of combustion and/or suppress the oxygen supply
    - Fire stages:
        - **Stage 1**: incipient stage: at this stage, there is only air ionization and no smoke
        - **Stage 2**: smoke stage: smoke is visible from the point of ignition
        - **Stage 3**: flame stage: this is when a flame can be seen with the naked eye

        - **Stage 4**: heat stage: at stage 4, there is an intense heat buildup and everything in the area burns
    - Fire extinguisher classes:
        - **Class A**: common combustibles
        - **Class B**: liquids
        - **Class C**: electrical
        - **Class D**: metal
        - **Class K**: cooking material (oil/grease)
    - Four main types of suppression:
        - **wet pipe system**: (AKA closed head system): is always filled with water; water discharges immediately when suppression is triggered
        - **dry pipe system**: contains compressed inert gas
        - **pre-action system**: a variation of the dry pipe system that uses a two-stage detection and release mechanism
        - **deluge system**: uses larger pipes and delivers larger volume of water
    - Note: Most sprinkler heads feature a glass bulb filled with a glycerin-based liquid; this liquid expands when it comes in contact with air heated to between 135 and 165 degrees; when the liquid expands, it shatters its glass confines and the sprinkler head activates
- 3.9.9 Power (e.g., redundant, backup)
    - Consider designing power to provide for high availability
    - Most power systems have to be tested at regular intervals
    - As part of the design, mandate redundant power systems to accommodate testing, upgrades and other maintenance
    - Additionally, test failover to a redundant power system and ensure it is fully functional
    - The International Electrical Testing Association (NETA) has developed standards around testing power systems
    - Battery backup/fail-over power (including UPS/generators):
        - this is a system that collects power into a battery but can switch over to pulling power from the battery when the power grid fails
        - generally, this type of system was implemented to supply power to an entire building rather than just one or a few devices

3.10 Manage the information system lifecycle (OSG-10 Chpt 10)

- The Information System Lifecycle: the entire lifespan of a system, from initial concept to eventual decommission, which includes the components below; note that this Information System lifecycle is very similar (with the exception of integration) to the Software Development Lifecycle (SDLC):
    - Initiation/Requirements
    - Architecture & Design
    - Development
    - Testing (note verification and validation are part of testing)
    - Release/Deployment
    - Operations/Maintenance
    - Retirement/disposal
- 3.10.1 Stakeholders needs and requirements
    - Focused on understanding stakeholder needs and expectations, this initial phase is about ensuring the new system will meet the needs of the people that use it, and that there is a communicated and agreed upon understanding of those requirements
- 3.10.2 Requirements analysis
    - This is a detailed analysis of the functional and nonfunctional requirements, ensuring that goals of the system and the organization are in alignment, as well as an analysis of the requirements to understand any associated risks
- 3.10.3 Architectural design
    - The overall structure including components and integration points are now defined, and a blueprint of the system can now be used in development; also in this phase controls are prescribed to mitigate the previously identified risks
- 3.10.4 Development/implementation
    - Properly develop and implement the system; in this phase system development takes place, including hardware configuration and component integration
- 3.10.5 Integration
    - This phase includes integration testing of the components in the system, as well as testing integration with external systems as part of the development process

- 3.10.6 Verification and validation
    - The system undergoes system testing, including verification and validation of functionality and components, preparing for go-live
- 3.10.7 Transition/deployment
    - Once the system has passed unit and system testing, the system is deployed for in-production use; this phase includes migration of dev to prod environments, and may also include migration from legacy to the new system
- 3.10.8 Operations and maintenance/sustainment
    - The system is in operational, day-to-day use in this phase, which includes on-going typical system monitoring, maintenance and patching, change management, configuration management, system backups, and disaster-recovery testing for the system in production
- 3.10.9 Retirement/disposal
    - At some point, the system will be retired once it has completed its useful purpose and the lifecycle will be completed

Also see Understanding CISSP Domain 3: Security Architecture and Engineering - part 1, part 2, and part 3 on my blog, The Cyber Leader (note that some articles require a subscription)